CricketMind

A Multi-Modal Multi-Agent System for Cricket Analytics

Thesis Architecture Draft

1. The Core Concept

The Problem

Modern cricket analytics is heavily siloed. Visual analysis (e.g., coaches reviewing batting technique on video) and statistical analysis (e.g., data scientists analyzing strike rates and match-ups) are performed completely independently. No existing artificial intelligence system automatically bridges the gap between what a cricket shot *looks like* (kinematics) and what its *historical outcome* is (statistics). Furthermore, attempting to solve complex sports queries using a single Large Language Model (LLM) often results in hallucinations and loss of reasoning context.

The Solution: CricketMind

CricketMind is a Multi-Modal, Multi-Agent System (MAS) built on the LangGraph framework. Instead of relying on a single AI brain, the system delegates complex analytical tasks to a decentralized ”team” of specialized AI agents.

The system takes two inputs simultaneously:

1. A natural language query (e.g., *”What is the tactical weakness of this batsman against spin?”*).
2. A raw video clip of the batsman in action.

2. System Architecture Diagram

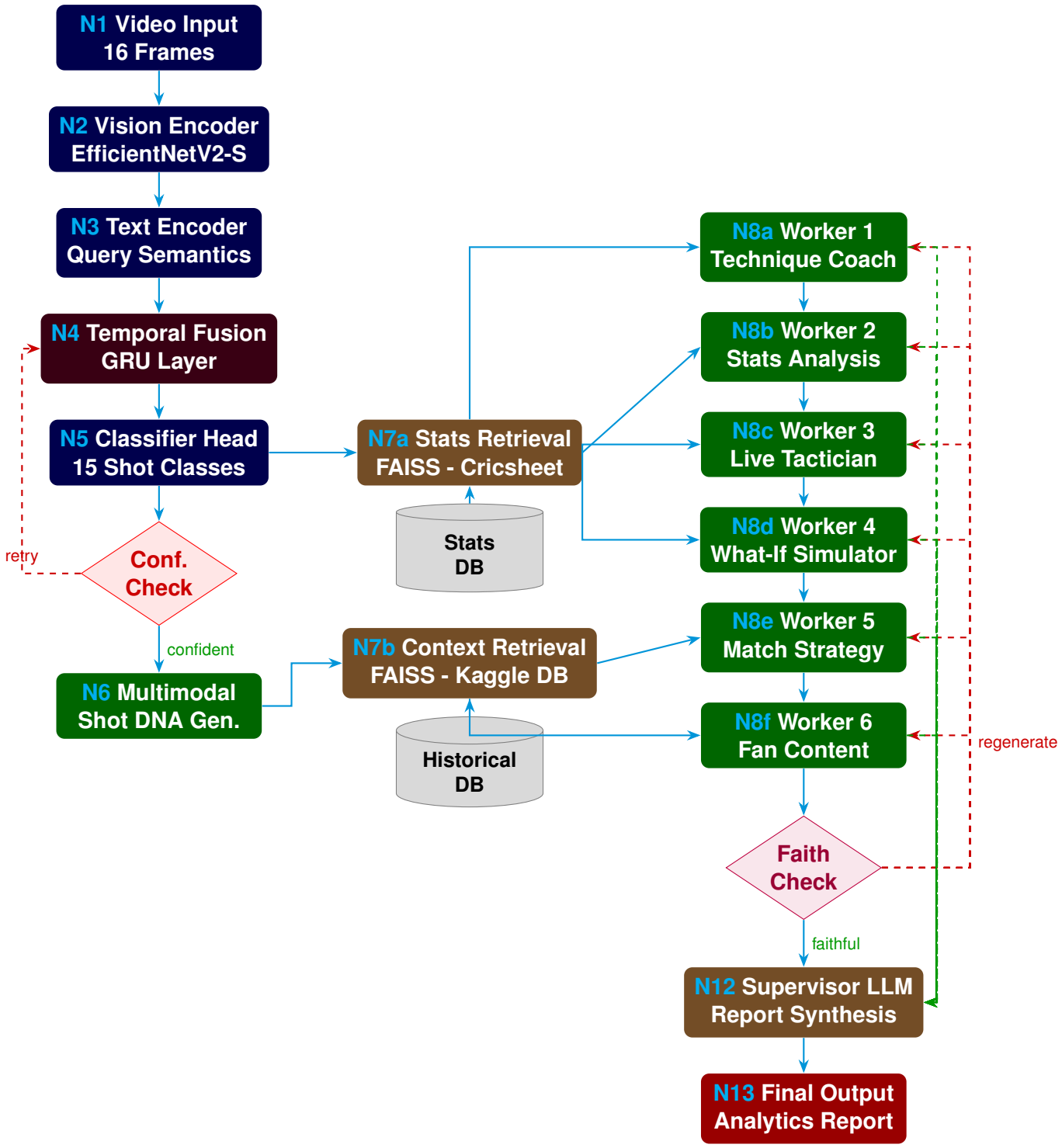

Figure 1. Full CricketMind LangGraph architecture showing visual processing, retrieval, and multi-agent coordination loops.

3. Node-by-Node Explanation

To understand the system flow, we trace a concrete user query:
Running Example Query: *"Compare Virat Kohli's Cover Drive vs his Pull Shot against fast bowlers in the powerplay."*

A. Vision & Text Processing Layer (Dark Blue Nodes)

- **N1: Video Input (16 Frames):** The user uploads a video clip of Kohli batting. The system samples 16 frames.
- **N2: Vision Encoder (EfficientNetV2-S):** The system extracts the spatial features (where the bat is, posture, ball position) from each frame.
- **N3: Text Encoder:** The query is embedded so the system knows to look specifically for "fast bowlers" and "powerplay" contexts.
- **N4: Temporal Fusion (GRU Layer):** Tracks the sequence of the 16 frames to understand the continuous motion of the bat swing.
- **N5: Classifier Head:** The model classifies the video.
  Example Output: 92% probability of "Cover Drive" (Class 1).

B. Validation & Grounding (Red & Green Nodes)

- **Conf. Check (Red Diamond):** If the classifier output is below a set threshold (e.g., 70%), it triggers the red dashed agentic loop to retry (perhaps requesting an alternate camera angle or applying data augmentation).
- **N6: Multimodal Shot DNA Gen:** The system fuses the "Cover Drive" classification with visual posture metrics, readying it for the stats engine.

C. Retrieval Layer (Brown Nodes)

- **N7a: Stats Retrieval (FAISS - Cricsheet):** The system queries the ball-by-ball database for "Kohli + Cover Drive + Powerplay + Pace".
- **N7b: Context Retrieval (FAISS - Kaggle):** Retrieves broader historical conditions, e.g., "Kohli's overall strike rate at this specific venue."

D. The LangGraph Worker Agents (Dark Green Nodes)

The supervisor dynamically routes the data to the necessary specialized workers:

- **N8a: Worker 1 (Technique Coach):** Notes that Kohli's visual weight transfer is excellent on the cover drive, yielding high efficiency.
- **N8b: Worker 2 (Stats Analysis):** Calculates that his Strike Rate is 142 on Cover Drives but only 112 on Pull Shots in the powerplay.
- **N8c: Worker 3 (Live Tactician):** If this were a live match, this agent would recommend bowling short to Kohli based on the pull shot data.
- **N8d: Worker 4 (What-If Simulator):** *Counterfactual Example:* "What if the bowler bowls 6 back-of-length deliveries?" The Monte Carlo simulation reveals a 40% chance of Kohli getting out trying to pull.

- **N8e: Worker 5 (Match Strategy):** Scales these insights to team-level strategy.
- **N8f: Worker 6 (Fan Content):** Converts the raw data into an engaging tweet or broadcast graphic.

E. Final Synthesis (Purple & Brown Nodes)

- **Faith Check (Purple Diamond):** Before showing the user the answer, the system checks for hallucinations. If Worker 2 says Kohli's average is 999, the Faith Check fails and triggers the "regenerate" loop (red dashed line) forcing the worker to recalculate.
- **N12: Supervisor LLM:** Taking the successful outputs from the workers (green dashed lines), it synthesizes a final, beautifully formatted report.
- **N13: Final Output:** The user receives the answer: *"Kohli's Cover Drive is statistically superior against pace in the powerplay (SR 142 vs 112). Our simulations show opposition bowlers should exploit his pull shot by bowling short..."*